1. Analyze two GenAI programs:

Claude Sonnet 5.5 | Thinking | High

Moonshot AI's Kimi K2.5 multimodal | Thinking

2. How you accessed them:

https://claude.ai/ | Claude Sonnet 5.5 | Thinking

https://www.kimi.ai/agent?chat_enter_method=change_model | Kimi K3 | Thinking | Max | Context Window: Standard

3. Prompt used

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
- Each digit from 1 to 9 must appear exactly once in each row.
- Each digit from 1 to 9 must appear exactly once in each column.
- Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness).
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that
  It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire Grid.

- That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively

try all possibilities.

- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
- The puzzle file name (e.g., puzzle1.txt)
- The puzzle from the file printed out.
- The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

I will also give each AI each puzzle.txt which means that they will be see the layout of all of the puzzle.txt which means that they should be able to see all of the possibilities given

The two GenAI examples include one other difference.

a. What is the difference?

Despite both of them using recursive solutions, Claude uses a Minimum Remaining Value(MRV) because it reduces the amount of possibilities of finding a valid number while Kimi K2.5 Flash Thinking uses a more rudimentary approach where vals and tries them sequentially and if that attempt doesn't produce a solution it rests the cell and tries at it again.

Analyze the difference in terms of:

b. Execution time

Claude Does have the edge on this, because it uses MRV this means that Claude's code does run faster than KIMI K2.5. Which means that claude has the advantage over KIMI K2.5.

c. Space complexity

Claude's solution stores a completely new board and stores it ina vector and copies it into a grild which means that it stores an additional 81 integers where Kimi does not do that and just prints the solution if it finds one.

d. Maintainability

Claude's code is more verbose and harder to read, although it has comments Kimi's code does more with less without less which means that it is easier to see what's going on however.

Kimi's code does store everything inside more Sudoku classes which means that if you wanted to refactor one part of the code it becomes much harder because it's already so interconnected. That means that Claude's code does have an advantage here because it's broken down into many different classes.

3. Which of the two GenAI code examples would you select? Why?

- I ended up liking KIMI K2.5 code more because I value readability over time saving because this is a recursive approach and a brute force approach. Execution time doesn't matter in comparison because program is already inefficient due to how it's already written so I would not try to maximize execution time

4. How would you improve the following for the one you selected

a. Correctness

Well the code runs and it matches the correct answer from the modules. However, the code doesn't run if the code isn't just puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt and doesn't allow the user to adjust it. However, that problem is already baked into the prompt. So now I prompt the user to give me the files that it wants to solve so that the names of the puzzle txt files allowing for more variations of what it could be

I added an error message if there is a String or messing col that isn't supposed to be there. Indicating that anything is wrong the program does not run that and notifies the user of there is anything wrong with the file.

b. Execution time

Execution time could technically be improved however it's not worth it because the approach is already inefficient.

c. Space complexity

This approach already uses

d. Maintainability

No comments on any of the code, which will be fixed when I add comments on large chunks of code